

Computer Graphics: Lecture 8

The math we all forgot

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](#)

Welcome Back!

Last time we learned about textures:

- How they are stored
- How they are sampled
- How to create them
- How to load them
- How to put them in a bind group
- How to make that bind group available to a shader
- How to use `textureSample` in the shader.

This time

We're going to review all the important math that we forgot:

- π and τ ¹
- Trigonometric functions (especially $\sin$, $\cos$, $\tan$, and inverses)
- The Pythagorean theorem
- The law of cosines
- The dot product
- Complex numbers and quaternions (brief)

“Oh, yuck.” right?

¹You might not have learned this one but once you see how useful it is you'll wish you had!

Not so fast

One of the reasons why math can be so challenging in school is that we often learn it way before we actually need it.

This is logical: math is useful no matter what, whether it be improving general problem-solving ability or describing a specific technique.

However, it makes it hard for us to force ourselves to do it unless there is a clear goal in sight.

I'm hoping that the same goals that drove you to take this class will help you want to learn these topics. I will pair each math topic with something it will help you do in a game or simulation!

Dimensionality

You mainly learned trig in 2 dimensions.

It's kind of surprising that everything you learned pretty much works the same in 3.

The pythagorean theorem works in 3-dimensions, as does the dot product.

I'm going to be using 2-D exampls to keep things easy to illustrate, but just be aware: scaling up isn't a problem.

Etymology

First of all, what does “trigonometry” mean. What are the Greek roots?

[class?]

Etymology (2)

Ancient Greek:

- Trígōnon meant triangle
- Métron meant measure

So, trigonometry is measuring triangles.

Cool. Who cares? [Why are triangles important at all?]

Why triangles are important

There are two ways that people frequently think of directions:

1. By grid reference: “go to D4” or “go 4 miles east and 3 down”
2. By angle + distance: “face southeast and go 5 miles”

The first one uses *rectangular coordinates*. That is, the coordinates are the lengths of sides of a rectangle with a known anchor point.

The second uses *circular* or *angular* coordinates. They are described by an direction (angle) and a distance (radius), therefore defining a circle.

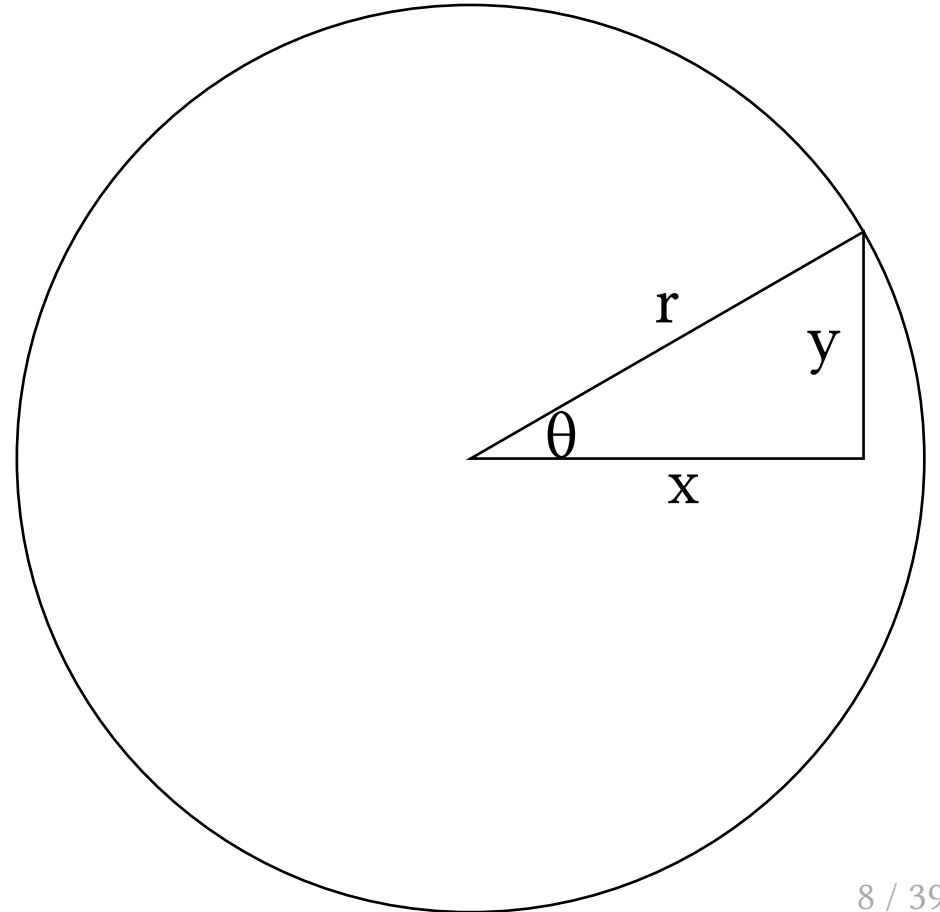
In games and simulations, we use both, often interchangeably. We want to position something in space, and rotate it to face a direction, ideally in one operation.

Why triangles are important (2)

Right triangles relate rectangles to circles.

That is, we can go back and forth from a pair of side lengths to an angle + radius.

The legs of the right triangle are the rectangular coordinates, and the inner angle and hypotenuse length are the circular coordinates.

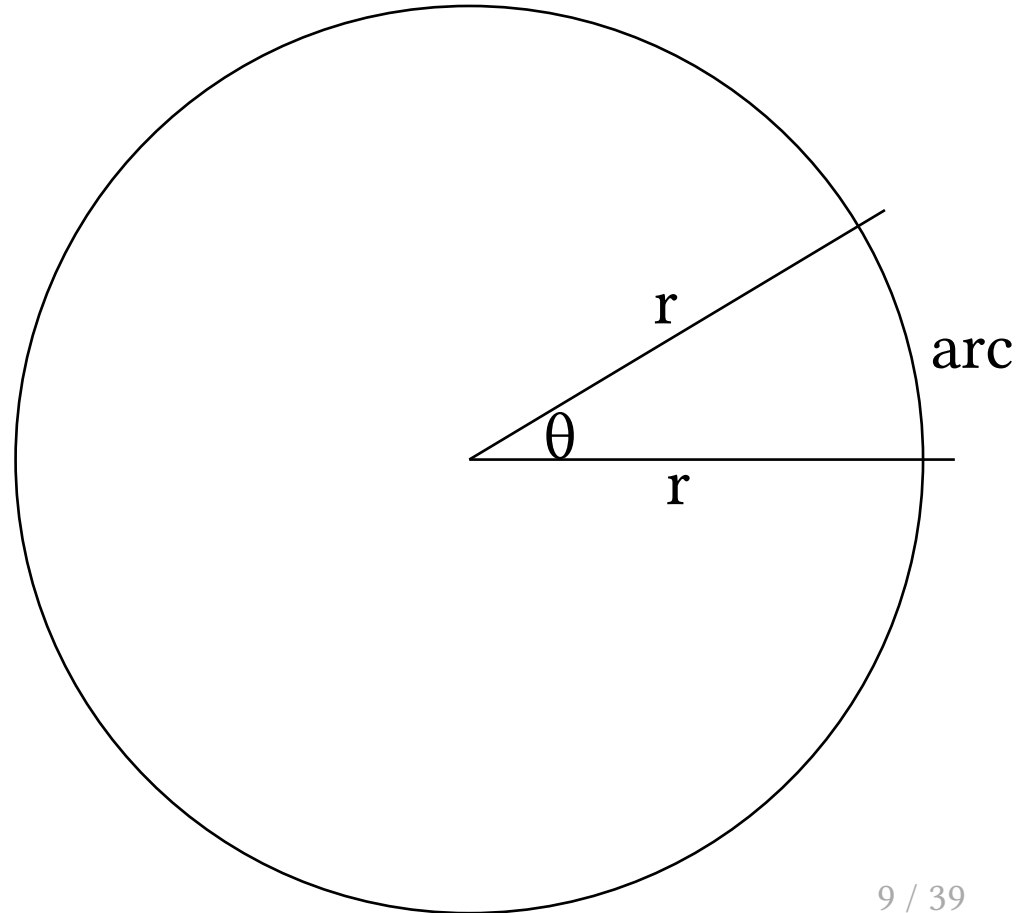


What even is an angle?

To me, the most straightforward definition of the measure of an angle, is that, for a sector of a triangle, the angle is the ratio of the arclength to the radius.

If the radius is 1, the distance around the outside is equivalent to the angle in radians.

That is: $\theta = \frac{\text{arc}}{r}$



Measuring angles

Angles are technically unitless (a distance over a distance), but we typically use a reference point to let the listener know how we wish to define the angle.

The most common are degrees and radians.

A radian is a length of 1, but referring to an angle. So “1 but going around the outside of the circle”.

Because the unit circle has a circumference of 2π , a full rotation is 2π (radians).

A degree is $\left(\frac{1}{360}\right)^{\text{th}}$ of 2π ., or 0.017453292519943295 radians

Radians are awkward

It is an unfortunate historical fact that π refers to the area of the unit circle rather than its circumference (which is 2π).

What if we defined a new symbol, τ , as being equal to 2π ?

A rotation of τ means a full rotation around the circle. Therefore, we can define rotation in “turns” instead of radians or degrees.

For example, a rotation of $40\%\tau$ means 40% of the way around a circle. What is that in radians? $.4\tau = .4 \times 2\pi$, which is 2.51ish.

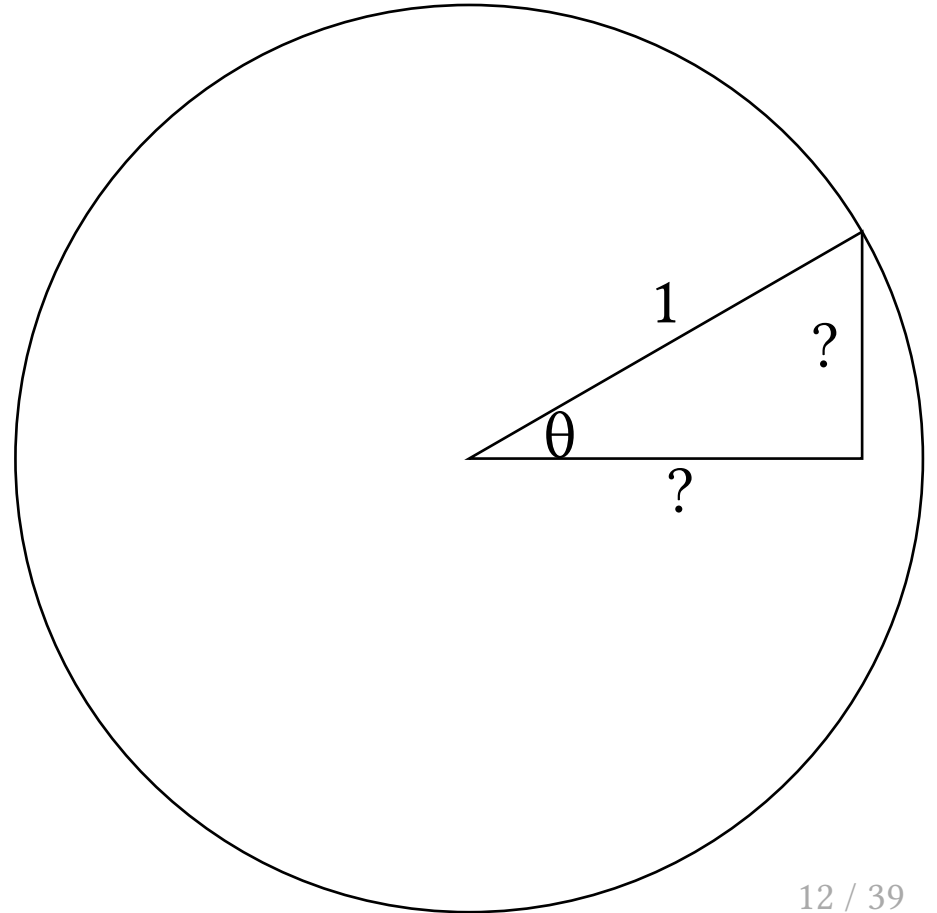
I prefer to define my rotations in turns. To convert to radians so you can use trig functions: multiply by τ .

Converting from angular to rectangular

So, we have an angle, which, together with the radius, can define a specific point on the outside of the circle.

Let's make it the unit circle: now the radius is 1.

How do we convert to rectangular coordinates?



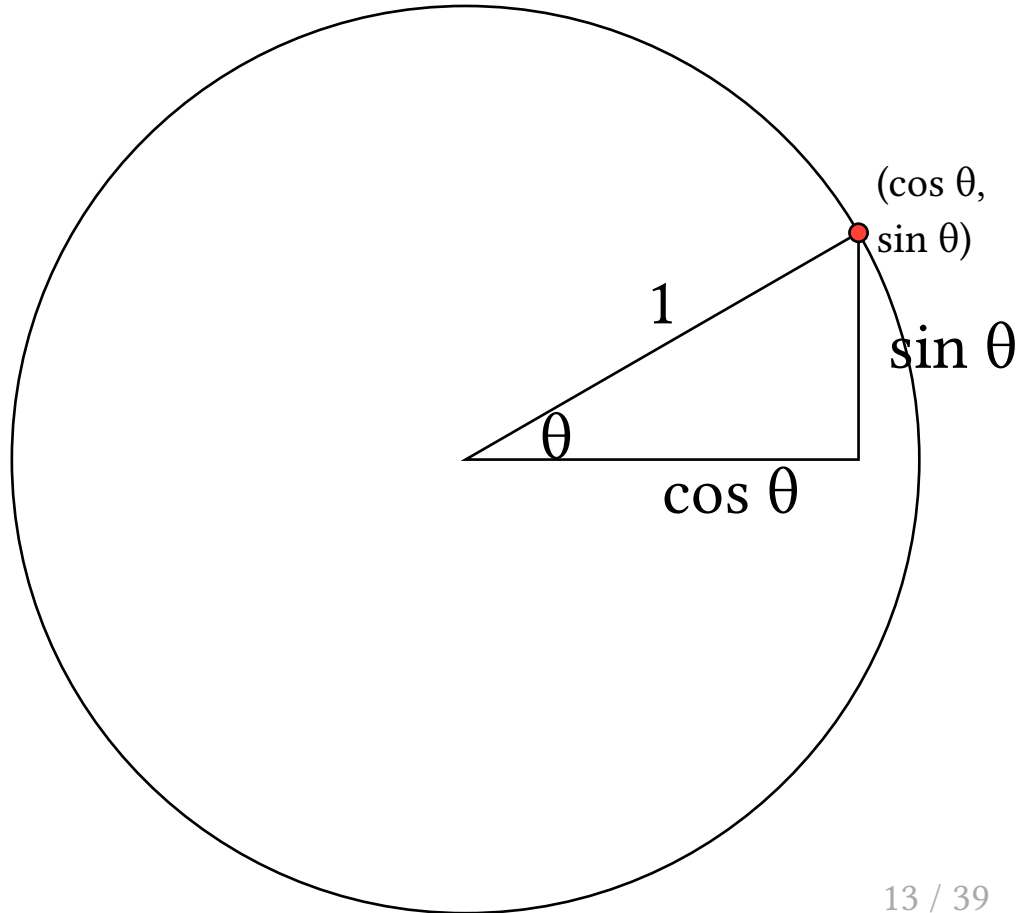
Converting from angular to rectangular (2)

This is what sine and cosine are for.

The sine of an angle is the length of the y leg.

The cosine is the x leg's length.

They are defined assuming a hypotenuse of 1, so if the circle is not a unit circle, we scale the lengths by the hypotenuse.



Concrete example

Let's make this immediately concrete:

We want a particle effect to circle around the player's character model. Each particle circles at a rate of 1 rotation every 2 seconds.

The initial position of the particle is a vertex attribute. The global time is given in a bind group.

Given a particle, what is its angle of rotation relative to the player it is orbiting?

[let's think about it]

Answer

1 rotation every 2 seconds means an angular speed of $\tau / 2$ seconds.

We are given the time, multiply it by the angular speed to solve for angle. So if we're at time 1.5 seconds, then the angle is $1.5/2 = 75\%$ of a turn ($75\%\tau$ radians)

Questions?

What about $\tan$?

There's one more trigonometric function that we always see that goes with the other two: the tangent.

It's name comes from the fact that it describes the length of a particular tangent line, but in my opinion, that is a less interesting fact about it.

A *more* interesting (to me) fact about it is that it is the ratio of sine over cosine. Or the vertical component over the horizontal component.

Why would that be interesting?

The tan is the slope of the triangle

Remember rise over run?

That's just sine over cosine.

Knowing the tangent, you immediately know the slope of the triangle, which is a good way of describing the sine and cosine together

Going backwards

Let's say we know the rectangular position of something, but we want to know its angle and distance.

Application: we want to give an objective marker to the player. So tell them which way to turn and how many distance units to travel.

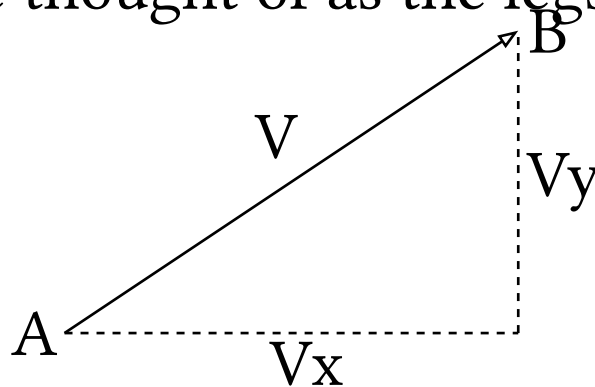
We know the player's position and the objectives position, and we've decided that east is in the $(1, 0)$ direction and corresponds to an angle of 0.

Going backwards (2)

First, we can create a vector by subtracting two points. $B - A$ can be represented as a vector whose tail is at A and whose head touches B . That is, if added to A , the result is B .

This vector in two dimensions has two coordinates, an x and a y .

Those coordinates can be thought of as the legs of a triangle:



Going backwards (3)

To get the distance of a vector (the arrow that points to the destination), we use the pythagorean theorem.

That is, if V_x and V_y are the legs of a right triangle, and $|V|$ is the length of its hypotenuse, then $V_x^2 + V_y^2 = |V|^2$. Or, solving for $|V|$:

$$|V| = \sqrt{V_x^2 + V_y^2}$$

So that's how far away the objective is.

But what about its angle?

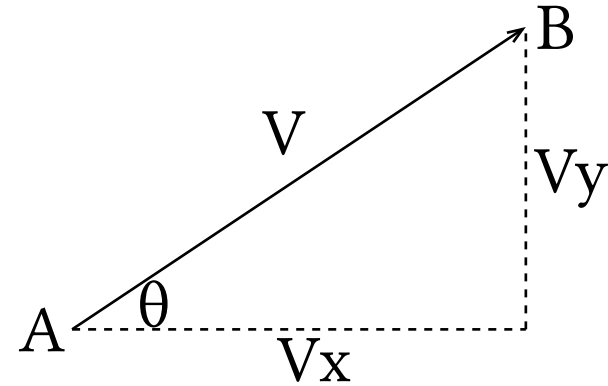
Going backwards (4)

We want to solve for θ

We know the following facts¹:

- $\cos(\theta) = \frac{V_x}{|V|}$
- $\sin(\theta) = \frac{V_y}{|V|}$
- $\tan(\theta) = \frac{V_y}{V_x}$

No matter what, we need to use an inverse trig function here...



¹These facts come from the fact that when the radius is 1, V_x is just cosine of theta. However, we have to scale by the radius. So if we want to solve for the cosine itself, we have to divide by the radius. Sine and tangent are similar.

Inverse trig functions

If we want to solve for theta, we need to know the angle that corresponds to one of the side lengths. This is the inverse of the trig functions.

We have several choices:

- $\theta = \arccos\left(\frac{V_x}{|V|}\right)$
- $\theta = \arcsin\left(\frac{V_y}{|V|}\right)$
- $\theta = \arctan\left(\frac{V_y}{V_x}\right)$
- $\theta = \text{atan2}(V_y, V_x)$

Which should we choose

Using `atan2`

You might not have seen it before, but `atan2` is almost always best.

It's a two argument tangent function. It takes the y and the x.

The reason it exists is that it handles all the little edge cases:

- What if either or both arguments are zero?
- What if either or both arguments are infinity?
- Most important: what if the point is outside the right semicircle

Also, the `arccos` and `arcsin` are very sensitive near zero.

So use `Math.atan2` in Javascript/Typescript.

Summary so far

So, if we want to use an angle and distance to generate some points (like particles that orbit a player), we use sine, cosine, and multiplication.

If we want to go backwards, we use the pythagorean theorem and `atan2`.

But what do we do if we want to know the angle between two arbitrary vectors? Not the angle from the x axis, but the angle between them?
Stay tuned...

Questions?

Arbitrary angles

Let's say we're making a horror game where it's dangerous to look at enemies, or maybe just a spooky enemy that freaks out if you look at it.¹

We have the position of the enemy and the position of the player like before, but that information is not enough. We also need the direction the camera is pointing.

We could compute the angle between each vector and the x axis using inverse trig functions, and then subtract them.

However, there's an easier way...

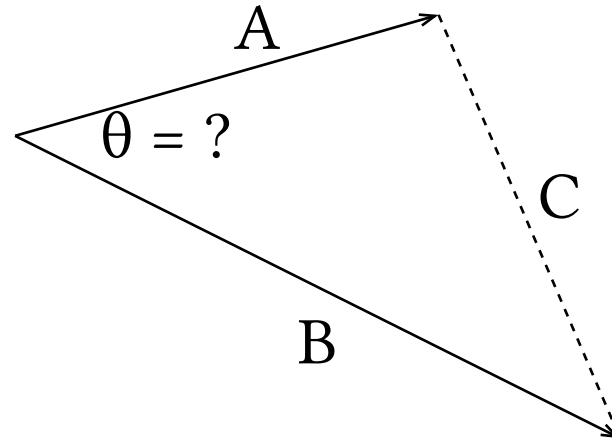
¹I have the Amnesia games and Endermen from Minecraft in mind here, but feel free to suggest more examples

Arbitrary triangles

Imagine the vectors form a triangle like this one.

We can only use the trig functions on right triangles, and this isn't one.

But, there is a useful trigonometric law we can use: **the law of cosines**



The law of cosines

The law of cosines is useful to solve for the angle of an arbitrary (not-necessarily right) triangle. It states:

$$|C|^2 = |A|^2 + |B|^2 - 2|A||B| \cos(\theta)$$

It's a more general case than the pythagorean theorem: the third side length depends on how wide the angle is. Let's see why this is useful:

$$|A - B|^2 = |A|^2 + |B|^2 - 2|A||B| \cos(\theta)$$

$$\begin{aligned} |A - B|^2 &= (A_x - B_x)^2 + (A_y - B_y)^2 \\ &= A_x^2 - 2A_x B_x + B_x^2 + A_y^2 - 2A_y B_y + B_y^2 \end{aligned}$$

The law of cosines (2)

Now let's expand the magnitudes of A and B alone:

$$|A|^2 = (A_x^2 + A_y^2), |B|^2 = (B_x^2 + B_y^2)$$

Putting it back together:

$$\begin{aligned} A_x^2 - 2A_xB_x + B_x^2 + A_y^2 - 2A_yB_y + B_y^2 = \\ A_x^2 + A_y^2 + B_x^2 + B_y^2 - 2|A||B|\cos(\theta) \end{aligned}$$

All of the squared magnitudes and the -2 factors cancel. We're left with:

$$A_xB_x + A_yB_y = |A||B|\cos(\theta)$$

The dot product

If A and B are unit vectors (meaning, length one), we now have a very fast way to calculate a cosine: $A \cdot B = A_x B_x + A_y B_y = \cos(\theta)$ This calculation is called the **dot product**.

What's so great about this? If the two vectors are the same (and unit length), their dot product will be 1. If they are perpendicular, it will be 0. If they are exactly opposed, it will be -1.

The cosine between two vectors tells us “how aligned” they are.

The dot-product is therefore one of the most common and useful operations in 3D graphics.

The norm

The dot-product is typically most useful when one or both vectors is unit-length.

How do we make a vector unit length?

Divide it by its norm, which is the magnitude of the vector. We often write the unit length version of vector v as $\hat{v}$.

$$\hat{v} = \frac{v}{|v|}$$

Converting a vector to a unit-length vector is called **normalization**.

Solving our question from before

So how do we know if we're looking at the enemy?

Make a vector from our player position to the enemy $v = E - P$.

Take the dot product between the unit-length version of that vector and the point that we're looking at (we'll call it 'c', for "camera"). If that value is close enough to 1, we can jump-scare the player:

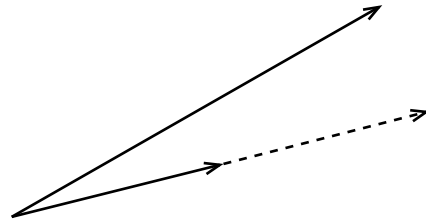
$$\hat{v} \cdot \hat{c} \geq \text{jump-scare point}$$

We don't need the actual angle: the dot product is good enough. The camera vector is usually always unit length, so this is easy to do.

Questions?

Projection

The dot product does something else cool: it calculates the projection of one vector onto another:



That is, the longer vector dotted with the unit length vector gives the length of the whole dotted vector (including the part covered up by the unit vector): the shadow.

To project onto non-unit length vectors, normalize: $\text{proj}(a, b) = a \cdot \frac{b}{(|b|)^2}$

Projection (2)

Why projection?

It's often useful to project a line onto a plane, for example, in an RTS game where you want to draw an arrow showing where a unit is moving.

If you have a normal vector (a vector pointing out perpendicular to a surface), you can check the projection of a vector to a point to the normal vector. Its direction will tell you which side of the surface it is.

Speaking of normal vectors...

Cross products

How do we get a vector that is perpendicular to two other vectors?

Better question: *why* do that at all?

The main reason has to do with camera controls. Cameras are often defined by a forward vector (pointing where you're looking) and an up vector (pointing up, so you know which way the camera is rolled).

The cross product is how you can compute the right vector, so you can side-step right and left.

Cross products (2)

The cross product is defined like this:

$$a \times b = |a||b| \sin(\theta)n$$

Where n is a normal unit vector.

Notice the similarity between this and the dot product, but also notice the difference: the dot product is a scalar (a one dimensional value), whereas this the scalar in the expression above is being multiplied by n , a vector, resulting in a vector.

That's why the cross product is often called the “vector product” (as opposed to the dot product, which is known as the “scalar product”).

Cross products (3)

Earlier I stated that most of the concepts we learn will work in either 2 or 3 dimensions.

The cross product is an exception. It *only* is defined in 3 dimensions¹.

Here is its definition:

$$A \times B = \langle A_y B_z - A_z B_y, A_z B_x - A_x B_z, A_x B_y - A_y B_x \rangle$$

You can see how it's kind of the opposite of the dot product. It's every product *except* the pairwise ones, and we subtract instead of add.

¹or technically 7, too

Cross products (4)

The cross product is honestly kind of janky, but it is useful.

We will be using a linear algebra library to compute it for us.

It helps to remember what direction the result points, though. There are two directions it *could* point, but we will be using the right-hand rule:

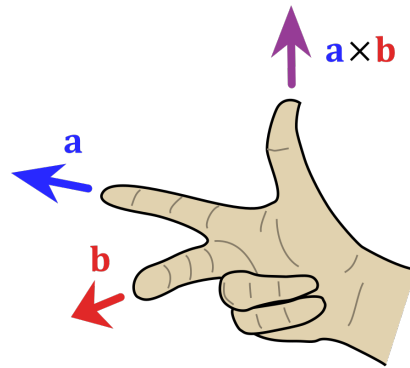


Figure 6: [Source](#), Acdx, [CC BY-SA 3.0](#), via Wikimedia Commons

Odds and ends

There's a couple of odds of ends we can cover if there's time:

- Complex numbers: what does i mean? It's the number that when you multiply it it means you rotate by 90 degrees: that's why if you do it twice you go from 1 to -1 . (i.e., $i^2 = -1$). But who cares?
- quaternions: we do. Because the extension of complex numbers into 3D is called a quaternion. There are 3 imaginary dimensions and one real dimension, and they encode scaling or rotation. Interestingly, they do it much faster than matrices do, and they are therefore quite useful to us for specific purposes.

Next class

Please [read this article](#) or watch the video linked in it: mandatory viewing.

It will explain matrices in a very intuitive way. And we are going to need that for next time!

You might also want to [read ahead](#) in the book.

Questions?